Online Continual Learning for Control of Mobile Robots

Andriy Sarabakha
School of Electrical and Electronic Engineering
Nanyang Technological University
Singapore
andriy.sarabakha@ntu.edu.sg

Zhongzheng Qiao
ERI@N, Interdisciplinary Graduate Programme
Nanyang Technological University
Singapore
qiao0020@e.ntu.edu.sg

Savitha Ramasamy
Institute of Infocomm Research (I2R)
*Agency for Science, Technology and Research (A*STAR)*
Singapore
ramasamysa@i2r.a-star.edu.sg

Ponnuthurai Nagaratnam Suganthan
KINDI Center for Computing Research
Qatar University
Qatar
p.n.suganthan@qu.edu.qa

2023 International Joint Conference on Neural Networks (IJCNN) | 978-1-6654-8867-9/23/$31.00 ©2023 IEEE | DOI: 10.1109/IJCNN54540.2023.10191188

***Abstract*—This work presents a novel approach which integrates deep learning, online learning and continual learning paradigms for adaptive control for robotic systems. Deep learning allows generalising knowledge about the robot, while online learning can adapt to variable operating conditions, and continual learning enables remembering previous knowledge. The proposed method approximates the inverse dynamics of the robot, which is formulated as a regression problem. With a minimum knowledge of the robot's dynamics, the proposed method shows its capability to reduce tracking errors online by continuously learning and compensating for internal and external changing conditions. Furthermore, the simulation results show that the proposed approach with online continual learning improves the control performance of ground and aerial mobile robots.**

***Index Terms*—continual learning, online learning, mobile robotics**

I. Introduction

Modern machine learning techniques, particularly deep learning, have revolutionized the fields of robotics [1]. In recent years, deep neural networks (DNNs) have shown an impressive ability to learn complex and large models. At the same time, the development of increased autonomy for mobile robots has taken centre stage due to their ability to provide efficient solutions to dangerous, dirty and dull tasks [2]. In many applications, such robotic systems must be able to operate autonomously in uncertain environments with variable operating conditions [3]. In such conditions, *online learnability is a must rather than a choice*. However, generally, DNNs are learned, i.e. trained offline before deploying for a task, and not learning, i.e. improving online during the execution of a task. After deployment to an end device, such as the on-board computer, DNNs are typically unchanged until the next software update, making them inflexible to the changing environment. Most existing approaches assume that DNNs are trained in a batch mode that requires the entire training dataset to be available before the learning task starts [4]. This is unsuitable in many real-world applications, where data arrives sequentially in a stream and might be too large to be stored [5]. Therefore, *a more desired option is to learn the models in an online fashion*.

This research was supported by the NTU Presidential Postdoctoral Fellowship (award number 021820-00001). This research is part of the programme DesCartes and is supported by the National Research Foundation, Prime Minister's Office, Singapore under its Campus for Research Excellence and Technological Enterprise (CREATE) programme.

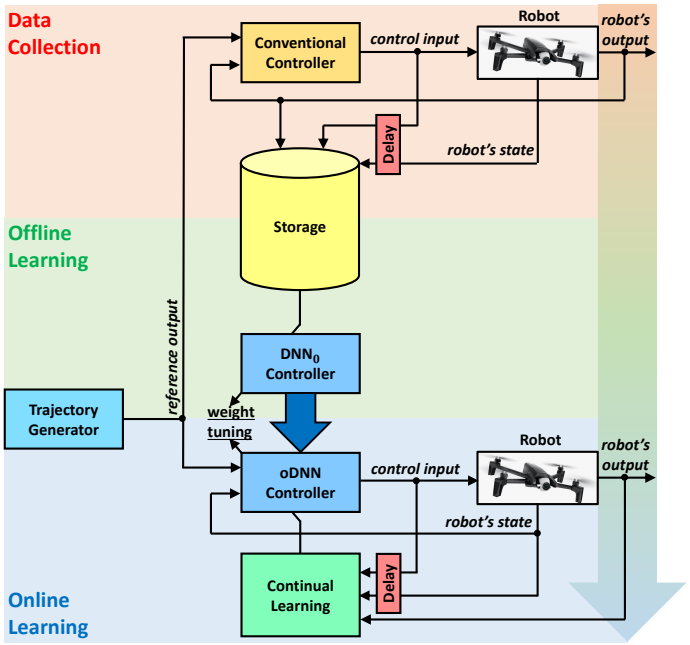


Fig. 1. Illustration of the proposed online continual learning-based control approach. Initially, a conventional controller executes a set of trajectories, and the input-output dataset is collected. Then, this dataset is used for the offline training of DNN_0 to learn the inverse dynamics of the robot. Finally, online DNN controls the robot and improves the performance by using CL.

In this work, we propose a novel online DNN-based control approach for efficient autonomous guidance of mobile robots with continual learning (CL) capabilities. The proposed method

approximates the inverse dynamics of the robot, which is formulated as a regression problem. The proposed learning approach is partitioned into offline pre-training and continual online training, as shown in Fig. 1. First, DNN is pre-trained on the collected input-output samples to approximate the inverse dynamics of the robot. Then, the pre-trained DNN controls the robot alone, and online continual learning is used to update the deep model. It is shown that the proposed controller is computationally effective for real-time operation, making it suitable for real-world control and navigation tasks. To the best of our knowledge, the contributions of this study can be summarized as: *(i)* for the first time, online CL is applied for the control of robotic systems; *(ii)* it was shown that the proposed approach is computationally adequate for real-time control applications; *(iii)* the proposed controllers are validated in simulation for the navigation of ground and aerial robots.

This manuscript is organised as follows. The existing works related to online learning and CL are outlined in Section II. The inverse control problem of a system is formulated in Section III. Section IV introduces the proposed method based on DNNs and online CL. Section V provides simulation results for a unicycle ground robot and quadrotor aerial robot. Finally, Section VI draws the conclusions and potential future work.

II. Related Works

Online learning represents a family of efficient techniques for updating models sequentially from data streams. In the literature, there are some attempts to achieve deep learning in an online setting [6]–[8]. In [6], shallow artificial neural networks were used in parallel with an error-driven controller to achieve online learning. In [7], DNNs rely on auxiliary Gaussian processes for online transfer learning to adapt to changing conditions. In [8], DNNs require expert knowledge embedded into a fuzzy logic system to monitor performance during online training. Typically, when Naïve online learning strategies are used to update the DNN in the non-stationary data stream, it can only remember the pattern learned from the latest distribution but forgets how to react in all the previous environments [9]. When data distribution is non-stationary, one challenge for training a model sequentially is the catastrophic forgetting problem – the phenomenon in which DNN tends to forget the knowledge learned from previous experiences when fine-tuning using the new data distribution. To overcome the catastrophic forgetting, a trade-off between rigidity, i.e. being good on old tasks, and plasticity, i.e. being good on new tasks, has to be found [10].

Research and development in CL are growing at an unprecedented speed, and several efficient CL techniques were proposed in the literature [11]–[13]. One of the first CL attempts, Learning without Forgetting (LwF) in [14], combines knowledge distillation and fine-tuning. One of the most popular CL methods, Elastic Weight Consolidation (EWC) in [15], slows down learning on certain weights based on their importance to the previously seen tasks. An end-to-end open-source library for CL, Avalanche, was recently released [16].

As a subfield of CL, online CL has attracted increased attention due to its property of real-time adaptation [17]. Experience Replay (ER) from [18] is a simple yet strong replay-based online CL baseline using reservoir sampling for memory update. It was shown that ER with a tiny memory size could surpass several specifically designed CL algorithms in the single-pass setting. In [19], different online CL scenarios are investigated in the image classification regime, and ER shows a competitive performance among a wide range of CL algorithms. Average Gradient Episodic Memory (A-GEM) in [20] proposes a single-pass setting to mimic the online learning scenario in which the learner observes training examples from the stream only once. It constrains the gradient direction to ensure that the average loss for all past tasks does not increase. In [21], an efficient and online version of EWC, called EWC++, is used to compute and update the Fisher information matrix. Besides using deep neural networks, an enhanced random vector functional link network-based online CL method without using extra memory is proposed in [22] and achieves effective performance in image classification challenges. However, most of the current online CL methods are proposed in the scope of classification problems [23]; while online CL for the regression tasks is greatly under-explored [24].

III. Problem Formulation

Nearly all physical systems are inherently nonlinear since most real-world relationships are nonlinear [25]. As a typical instance of a physical system, every robot is a nonlinear dynamical system. Moreover, every nonlinear dynamical system can be represented by its state-space model:

$$\begin{cases} \mathbf{x}[k+1] & = f(\mathbf{x}[k]) + g(\mathbf{x}[k])\mathbf{u}[k] \\ \mathbf{y}[k] & = h(\mathbf{x}[k]), \end{cases} \tag{1}$$

where $\mathbf{x} \in \mathbb{R}^{N_S}$ is the state of the system, $\mathbf{u} \in \mathbb{R}^{N_I}$ is the input to the system, $\mathbf{y} \in \mathbb{R}^{N_O}$ is the output from the system, $k \in \mathbb{N}_0$ is the discrete sampling time, and $f : \mathbb{R}^{N_S} \to \mathbb{R}^{N_S}$, $g : \mathbb{R}^{N_S} \to \mathbb{R}^{N_S} \times \mathbb{R}^{N_I}$ and $h : \mathbb{R}^{N_S} \to \mathbb{R}^{N_O}$ are system functions.

Definition 1: The vector of relative degrees $\mathbf{r} \in \mathbb{N}_0^{N_O}$ is the number of times one has to differentiate the output $\mathbf{y}$ to have at least one of the inputs $\mathbf{u}$ explicitly appear [26], i.e.

$$\arg\max_{\mathbf{r}_i} \frac{\partial}{\partial \mathbf{u}[k]} \left(h_i \left(f^{\mathbf{r}_i - 1} \left(f(\mathbf{x}[k]) + g(\mathbf{x}[k])\mathbf{u}[k] \right) \right) \right) \neq \mathbf{0}, \tag{2}$$

where $i \in \{1, \ldots, N_O\}$.

The system's input, state and output are related by

$$\mathbf{y}_i[k + \mathbf{r}_i] = h_i \left(f^{\mathbf{r}_i - 1} \left(f(\mathbf{x}[k]) + g(\mathbf{x}[k])\mathbf{u}[k] \right) \right). \tag{3}$$

If $\mathbf{y}$ is affine in $\mathbf{u}$, then (3) becomes

$$\mathbf{y}_i[k + \mathbf{r}_i] = \mathcal{F}_i(\mathbf{x}[k]) + \mathcal{G}_i(\mathbf{x}[k])\mathbf{u}[k], \tag{4}$$

where

$$\mathcal{F}_i(\mathbf{x}[k]) = h_i \left(f^{\mathbf{r}_i}(\mathbf{x}[k]) \right) \tag{5}$$

and

$$\mathcal{G}_i(\mathbf{x}[k]) = \frac{\partial}{\partial \mathbf{u}[k]} \left(h_i \left(f^{\mathbf{r}_i - 1} \left(f(\mathbf{x}[k]) + g(\mathbf{x}[k])\mathbf{u}[k] \right) \right) \right) \quad (6)$$

are decoupling functions. To track the desired output of the system $\mathbf{y}^* \in \mathbb{R}^{N_O}$, the control law can be written as in [27]:

$$\mathbf{u}_i[k] = \mathcal{G}(\mathbf{x}[k])^{-1} \left(\mathbf{y}_i^*[k + \mathbf{r}_i] - \mathcal{F}(\mathbf{x}[k]) \right). \quad (7)$$

If an accurate model of the robotic system in (1) exists, the inversion of the system can be precisely computed with (7). However, the robot's parameters might be difficult to estimate, e.g. moments of inertia, or might vary during the operation, e.g. center of gravity. Besides, it is challenging to predict external disturbances, e.g. wind gusts. In addition, measurements might come from small and noisy onboard sensors, e.g. inertial measurement unit (IMU). Thus, instead of the model in (1), a new unknown model has to be considered:

$$\begin{cases} \mathbf{x}[k] & = \tilde{f}(\mathbf{x}[k]) + \tilde{g}(\mathbf{x}[k])\mathbf{u}[k] + \mathbf{d}[k] \\ \mathbf{y}[k] & = \tilde{h}(\mathbf{x}[k]) + \mathcal{N}, \end{cases} \quad (8)$$

where $\tilde{f} : \mathbb{R}^{N_S} \rightarrow \mathbb{R}^{N_S}$, $\tilde{g} : \mathbb{R}^{N_S} \rightarrow \mathbb{R}^{N_S} \times \mathbb{R}^{N_I}$ and $\tilde{h} : \mathbb{R}^{N_S} \rightarrow \mathbb{R}^{N_O}$ are new system functions, $\mathbf{d} \in \mathbb{R}^{N_S}$ is the disturbance to the system and $\mathcal{N} \in \mathbb{R}^{N_O}$ is an additive noise. In real-life, $\tilde{f}$, $\tilde{g}$, $\tilde{h}$, $\mathbf{d}$ and $\mathcal{N}$ are unknown; consequently, the control law in (7) cannot be applied.

IV. Proposed Method

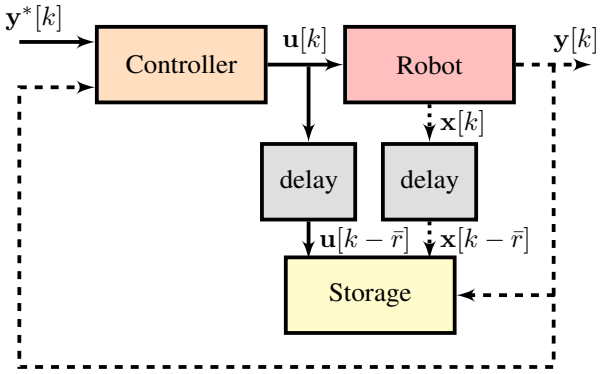
The approximation of the inverse dynamics in (8) can be formulated as a regression problem. To control the robot, we propose a CL-based control approach which learns the inverse dynamics of the robot online. The training of DNN is divided into two phases: offline pre-training and online adaptation, similar to [8].

A. Offline Phase

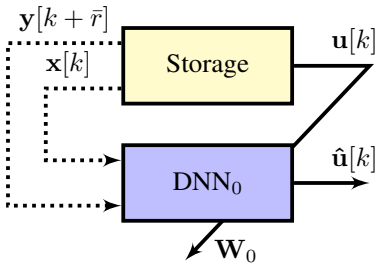
In the offline phase, a feed-forward DNN_0 learns the approximate inverse dynamics of the robot. In the control scheme, illustrated in Fig. 2a, a conventional controller, e.g. a proportional–integral–derivative (PID) controller, controls the robot, and labelled training samples are collected in storage memory. Each training sample $\mathcal{D}_0[k]$ consists of input $\mathcal{I}_0[k]$ and expected output $\mathcal{O}_0[k]$ pair:

$$\begin{aligned} \mathcal{D}_0[k] &=< \mathcal{I}_0[k], \mathcal{O}_0[k] > \\ &=< \{\mathbf{x}[k - \bar{r}], \mathbf{y}[k]\}, \{\mathbf{u}[k - \bar{r}]\} > . \end{aligned} \quad (9)$$

where $\bar{r} = \max_{\forall i \in [1, N_O]} \mathbf{r}_i$. The training of DNN_0 involves backpropagation, and the network weights $\mathbf{W}_0$ are updated, as shown in Fig. 2b. The pseudo-code of the offline pre-training is provided in Algorithm 1.



(a) Data collection by a conventional controller.



(b) Offline training of DNN_0.

Fig. 2. Block diagrams of the offline phase (solid lines represent calculated quantities, dashed lines represent measured quantities, dotted lines represent estimated quantities).

Algorithm 1: Offline pre-training of DNN_0.

Input: Network structure $\{N_{H_1}, \ldots, N_{H_L}\}$, number of samples n_S
Output: Pre-trained model parameters $\mathbf{W}_0$ of DNN_0
begin
  $k \leftarrow 1$
  while $k < n_S$ **do**
    Collect $\mathcal{D}_0[k]$ in (9)
    $k \leftarrow k + 1$
  end
  $\text{DNN}_0 \leftarrow \text{ConstructNetworkLayers}(N_{H_1}, \ldots, N_{H_L})$
  $\mathbf{W}_0 \leftarrow \text{ModelTrain}(\mathcal{D}_0)$
end

B. Online Phase

In the online phase, the DNN-based controller from Subsection IV-A drives the robot and, simultaneously, learns online how to improve the control performances by adjusting weights $\mathbf{W}$. Traditional online learning uses only a single incoming example at a time to calculate the gradients and backpropagate. However, such gradients are extremely biased and prone to fluctuate, resulting in unstable learning behaviour. Mini-batch stochastic methods have been proven to be more stable in backpropagation. We introduce two key components to implement online CL: *(i)* a cache $\mathcal{C}$ to store incoming samples from the data stream, and *(ii)* a buffer $\mathcal{B}$ to save historical

samples. The cache is a first-in-first-out (FIFO) queue of a fixed size s_C; while the buffer is a memory space of a fixed size s_B with a reservoir sampling strategy. To stabilize the learning in the initial stage, the buffer is initialised with random samples from $\mathcal{D}_0$ in (9). Each sample $\mathcal{D}[k]$ coming from the online stream consists of input $\mathcal{I}[k]$ and corresponding output $\mathcal{O}[k]$ pair:

$$\begin{aligned}\mathcal{D}[k] &=< \mathcal{I}[k], \mathcal{O}[k] > \\ &=< \{\mathbf{x}[k-\bar{r}], \mathbf{y}[k]\}, \{\mathbf{u}[k-\bar{r}]\} > .\end{aligned} \tag{10}$$

A reservoir sampling is used on the data stream, and randomly selected samples are saved into the buffer. When updating the model, it randomly retrieves a mini-batch $\mathcal{M}$ from the memory buffer and combines it with the incoming mini-batch to calculate the gradient to update. This approach not only gives superior performance over the existing techniques for CL, but it is also computationally very efficient. In our method, the cache is a mini-batch in the online learning setting. The cache size equals the batch size, an important parameter affecting the learning performance.

Remark 1: If the cache size is too small, the learning might be unstable; while if the cache size is too large, it will increase the update time and make the learning unable to follow the changes in the dynamics.

When the cache is completely filled with new samples, i.e. all the samples in the cache are replaced, the learning is triggered, and DNN is updated using the gradient calculated on the samples from the cache. When the data distribution is stationary, we can obtain an unbiased estimator of the exact gradient of the generalization error. Consequently, simple but effective online CL algorithms can be implemented, in which DNNs are jointly trained on samples from the current stream and samples stored in small episodic memory. We have adapted the following CL methods for online regression:

a) Experience Replay (ER) [18]: Due to its simplicity, ER can be implemented for online regression problems directly. Samples from the data stream are randomly selected and saved in the buffer using reservoir sampling. When a new mini-batch $\mathcal{C}$ is available, ER randomly retrieves a mini-batch $\mathcal{M}$ from the memory buffer $\mathcal{B}$ and trains the model with the combination of two mini-batches.

b) Learning without Forgetting (LwF) [14]: LwF is a regularization-based method which uses knowledge distillation. Using cross-entropy on the prediction space, LwF distils the knowledge to the current student model from the teacher model, which is preserved after learning the last task. To accommodate the online learning for a regression problem, we adapted the original LwF algorithm and the teacher update strategy. The distillation is conducted in the prediction space with the mean squared error (MSE)-based training loss. For a mini-batch of data $< \mathcal{I}, \mathcal{O} >$, the loss function is

$$\mathcal{L}(\mathcal{I}, \mathcal{O}) = \mathcal{L}_{\text{cur}}(\mathcal{I}, \mathcal{O}) + \lambda \mathcal{L}_{\text{old}}(\mathcal{I}, \mathcal{O}), \tag{11}$$

in which

$$\mathcal{L}_{\text{cur}}(\mathcal{I}, \mathcal{O}) = \|\text{DNN}(\mathcal{I}) - \mathcal{O}\|^2, \tag{12}$$

$$\mathcal{L}_{\text{old}}(\mathcal{I}) = \|\text{DNN}(\mathcal{I}) - \text{DNN}_{\text{old}}(\mathcal{I})\|^2, \tag{13}$$

λ is a hyperparameter to balance the stability and plasticity, and DNN_{old} is the saved teacher model. There are no task boundaries in the online data stream, so we change the triggering condition for updating the teacher model. The latest model replaces the teacher model after a fixed number of steps.

c) Average Gradient Episodic Memory (A-GEM) [20]: Unlike direct rehearsal, A-GEM leverages the memory samples to regularize the gradient when learning new knowledge. Solving an optimisation problem, it aims to ensure that the average loss for all past tasks does not increase. After calculating the gradient ∇ on the incoming mini-batch, a reference gradient ∇_{ref} is computed on a mini-batch randomly sampled from the memory buffer. If $\nabla^T \nabla_{\text{ref}} \geq 0$, ∇ is used to update the model directly; otherwise, ∇ is projected as:

$$\tilde{\nabla} = \nabla - \frac{\nabla^T \nabla_{\text{ref}}}{\nabla_{\text{ref}}^T \nabla_{\text{ref}}} \nabla_{\text{ref}}, \tag{14}$$

and, then, $\tilde{\nabla}$ is used to update the model.

d) Efficient Elastic Weight Consolidation (EWC++) [15]: As another canonical regularization method, EWC regularizes the learning based on the importance of the parameters. It penalizes the update of parameters which were important for the past tasks. This important factor is approximated by the diagonal of the Fisher information matrix $\mathbf{F}$. In our setting without task boundaries, the loss function for a mini-batch $< \mathcal{I}, \mathcal{O} >$ is

$$\mathcal{L}(\mathcal{I}, \mathcal{O}) = \mathcal{L}_{\text{cur}}(\mathcal{I}, \mathcal{O}) + \lambda \sum \mathbf{F}_j (\mathbf{W}_j - \mathbf{W}^*_{\text{old},j})^2, \tag{15}$$

in which $\mathcal{L}_{\text{cur}}$ is defined in (12), λ is a hyperparameter to control the regularization strength, and $\mathbf{W}^*_{\text{old},j}$ is the optimal value of the j^{th} parameter of the old model. To achieve the online settings, we implemented the online variant of EWC, EWC++ [21]. To avoid re-accessing historical data, $\mathbf{F}$ is updated online by using the exponential moving average:

$$\mathbf{F}[k] = \alpha \mathbf{F}_{\text{tmp}}[k] + (1-\alpha)\mathbf{F}[k-1], \tag{16}$$

in which α is a hyperparameter to control the degree of the weight decrease, and $\mathbf{F}_{\text{tmp}}[k]$ is calculated with the current batch of data. Similar to LwF, we update the saved model and replace $\mathbf{F}$ in (15) with $\mathbf{F}[k]$ after a given number of steps.

Independently on the CL technique used, for each sample, DNN computes a new control command:

$$\mathbf{u}[k] = \text{DNN}(\{\mathbf{x}[k], \mathbf{y}^*[k+\bar{r}]\}), \tag{17}$$

which is sent to the robot. The control scheme of the online training is shown in Fig. 3. The pseudo-code of online training is provided in Algorithm 2, where RandomRetrieve($\mathcal{A}$, n) is a function which returns a set of n random samples from set $\mathcal{A}$, and ReservoirSampling($\mathcal{A}$, $\mathcal{B}$) is a function which updates set $\mathcal{A}$ using reservoir sampling on elements from set $\mathcal{B}$.

Remark 2: Both ER and A-GEM require a memory buffer, while EWC++ does not need a memory buffer. At the same time, LwF has the option to be implemented with or without the memory buffer.

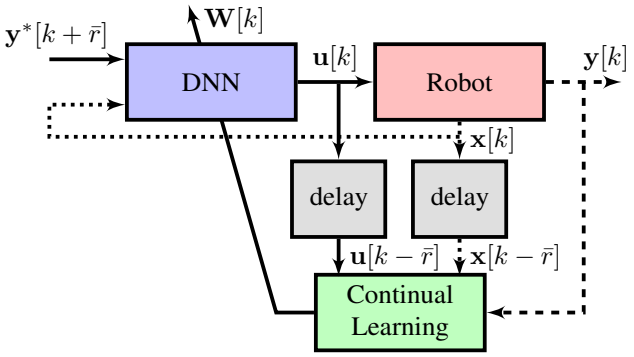


Fig. 3. Block diagrams of the online CL of DNN (solid lines represent calculated quantities, dashed lines represent measured quantities, dotted lines represent estimated quantities).

Algorithm 2: Online training of DNN.

Input: Pre-trained model parameters $\mathbf{W}_0$, cache size s_C, buffer size s_B
Output: Fine-trained model parameters $\mathbf{W}$
begin
  $\mathbf{W} \leftarrow \mathbf{W}_0$
  Cache $\mathcal{C} \leftarrow \emptyset$
  if *use buffer* **then**
    buffer $\mathcal{B} \leftarrow$ RandomRetrieve($\mathcal{D}_0$, s_B)
  end
  $k \leftarrow 1$ **repeat**
    Collect sample pair $\mathcal{D}[k]$ using (10)
    $\mathcal{C}[k \bmod s_C] \leftarrow \mathcal{D}[k]$
    if $k \bmod s_C = 0$ **then**
      if *use buffer* **then**
        mini-batch $\mathcal{M} \leftarrow$ RandomRetrieve($\mathcal{B}$, s_C)
        $\mathbf{W} \leftarrow$ ModelUpdate($\mathbf{W}, \mathcal{C} \cup \mathcal{M}$)
        $\mathcal{B} \leftarrow$ ReservoirSampling($\mathcal{B}$, $\mathcal{C}$)
      else
        $\mathbf{W} \leftarrow$ ModelUpdate($\mathbf{W}, \mathcal{C}$)
      end
    end
    $\mathcal{I}^*[k] \leftarrow \{\mathbf{x}[k], \mathbf{y}^*[k+\bar{r}]\}$
    Control signal $\mathbf{u}[k] \leftarrow$ DNN($\mathcal{I}^*[k]$)
    Send $\mathbf{u}[k]$ to the robot
    $k \leftarrow k+1$
  until *Stop*
end

V. Simulation Results

The methods presented in Section IV are validated and compared in two different environments: *(i)* numerical simulation on a unicycle robot and *(ii)* dynamical simulation on an aerial robot. The numerical simulation provides a transparent environment where it is possible to investigate all aspects of the proposed methods deterministically. On the other hand, dynamical simulation provides a complex real-world-like environment where more sophisticated physical effects can be studied.

A. Numerical Simulation: Unicycle

A unicycle is one of the simplest dynamical systems which possesses most of the characteristics of a typical mobile robot, such as nonlinear dynamics, coupled states, nonholonomic constraints, and underactuated control with translational and rotational actions. The absolute position of the unicycle $\mathbf{p} = \begin{bmatrix} x & y \end{bmatrix}^T$ is described by two Cartesian coordinates of its contact point with the ground; while the heading of the unicycle θ is described by an Euler's angle. Its forward velocity is v, and its angular velocity is ω. The unicycle motors are torque controlled by adjusting the driving torque τ_y and steering torque τ_z. The dynamical model of unicycle is derived in [28], and it can be reworked in form of (1), where $\mathbf{x} = \begin{bmatrix} x & y & \theta & v & \omega \end{bmatrix}^T$, $\mathbf{u} = \begin{bmatrix} \tau_y & \tau_z \end{bmatrix}^T$ and $\mathbf{y} = \begin{bmatrix} x & y & \theta \end{bmatrix}$.

The desired 2-dimensional position of the unicycle (x^*, y^*) is a time-based closed-loop ∞-shaped trajectory:

$$\begin{cases} x^*[k] & = 4\frac{\cos(2k)}{3-\cos(4k)} \\ y^*[k] & = -2\sqrt{2}\frac{\sin(4k)}{3-\cos(4k)}, \end{cases} \tag{18}$$

depicted in Fig. 4a. This trajectory contains most manoeuvres during a real ride, such as straight segments and curved sections with left-hand and right-hand turns. From (18), it can be calculated that the desired translational velocity is variable between $3.4\mathrm{m\,s^{-1}}$ and $5.7\mathrm{m\,s^{-1}}$ over the trajectory.

Remark 3: Due to the underactuation and nonholonomic constraints, the desired heading of the unicycle θ^* is a control parameter and is equal to

$$\theta^*[k] = \arctan 2(y^*[k] - y[k-2], x^*[k] - x[k-2]). \tag{19}$$

The unicycle's model is assumed to be a black-box system. Hence, only input-output data with some basic properties, such as the number of control inputs $N_I = 2$ and observed outputs $N_O = 3$, and the relative degree $\bar{r} = 2$, are known. To collect samples for the offline pre-training, the robot is controlled by a well-tuned PID controller. One million instances $\{x[k], y[k], \theta[k], \tau_y[k], \tau_z[k]\}$ are collected. To prepare the training samples, the linear velocity $v[k]$ and angular velocity $\omega[k]$ are numerically derived from position and orientation, respectively, and the following physics-informed considerations are made:

i) *position is translation-invariant*: all points can be translated with respect to the origin of the body frame, i.e., $\Delta x[k] = x[k+1] - x[k]$ and $\Delta y[k] = y[k+1] - y[k]$.
ii) *orientation is rotation-invariant*: all points can be rotated with respect to the orientation of the body frame, i.e., $\Delta\theta[k] = \theta[k+1] - \theta[k]$.
iii) *orientation is periodic*: $\pi \equiv -\pi$ in Lie algebra; thus, the heading $\theta[k]$ can be represented in quaternion-like form: $\{\cos(\theta[k]), \sin(\theta[k])\}$.

Consequently, the training pairs $\mathcal{D}_0[k] =< \{\cos(\theta[k]), \sin(\theta[k]), v[k], \omega[k], x[k+2] - x[k], y[k+2] - y[k], \theta[k+2] - \theta[k]\}, \{\tau_y[k], \tau_z[k]\} >$ are congregated according to (9). After some trials, the network architecture is selected to consist of two fully

connected hidden layers with 32 neurons in each layer besides one input and one output layer. Then, DNN_0 is trained on $\mathcal{D}_0$ using MSE loss and Adam optimizer.

Remark 4: Source code, collected datasets and trained models will be available at https://github.com/andriyukr/unicycle/tree/main/python.

After offline training, DNN through feedforward with input $\mathcal{I}[k] = \{\cos(\theta[k]), \sin(\theta[k]), v[k], \omega[k], x^*[k+2]-x[k], y^*[k+2]-y[k], \theta^*[k+2]-\theta[k]\}$ controls the robot online by generating the control signal $\mathcal{O}[k] = \{\tau_y[k], \tau_z[k]\}$ to reach the desired pose $\{x^*[k+2], y^*[k+2], \theta^*[k+2]\}$. At each iteration, the cache is updated with $\mathcal{D}[k] =< \{\cos(\theta[k-2]), \sin(\theta[k-2]), v[k-2], \omega[k-2], x[k]-x[k-2], y[k]-y[k-2], \theta[k]-\theta[k-2]\}, \{\tau_y[k-2], \tau_z[k-2]\} >$ according to (10). The parameters used in CL methods are tuned by trial-and-error and are as follows:

- LwF: $\lambda = 0.5$, cache size $s_C = 2$, buffer size $s_B = 1000$;
- ER: cache size $s_C = 8$, buffer size $s_B = 1000$;
- A-GEM: cache size $s_C = 16$, buffer size $s_B = 1000$;
- EWC++: $\lambda = 100$, $\alpha = 0.1$, cache size $s_C = 32$.

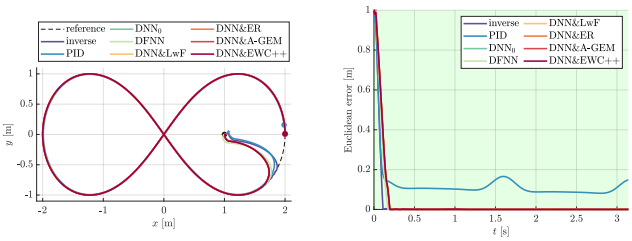
To show the efficiency of the presented DNN-based controllers enhanced with CL techniques, their performances are compared with the performances of the baseline DNN_0 controller, the exact analytical inverse calculated with (7), a well-tuned PID controller (used for collecting training samples) and deep fuzzy neural network (DFNN)-based controller from [29]. These controllers are tested in four scenarios: nominal dynamics, dynamics with internal uncertainties, dynamics with external disturbance and dynamics with noisy measurements. At the beginning of each experiment, the state of the unicycle is initialised as $\begin{bmatrix}10\text{m} & 0\text{m} & \pi\text{rad} & 0\text{rad}\,\text{s}^{-1} & 0\text{rad}\,\text{s}^{-1}\end{bmatrix}^T$.

1) Nominal Dynamics: In this scenario, the controllers are tested on a nominal unicycle on which the model-based controller, i.e. PID, was tuned and from which the training samples for the data-driven controller, i.e. DNN_0, were collected. Fig. 4a shows that all tested controllers can track precisely the desired trajectory. From Fig. 4b, it is possible to observe that a well-tuned PID controller still has a steady-state error due to its feedback-based nature and variable speed over the trajectory.

2) Dynamics with Internal Uncertainties: In this scenario, the controllers are tested on a unicycle with modified internal parameters; namely, the wheel's radius is halved, and the mass is doubled, corresponding to doubling the two moments of inertia. From Fig. 5, it is possible to observe that PID, DNN_0 and DFNN controllers cannot deal with changed parameters and their performances deteriorate over the trajectory.

3) Dynamics with External Disturbance: In this scenario, the controllers are tested on a unicycle affected by external disturbance; namely, the wind blows from the northwest at $3.4\text{m}\,\text{s}^{-1}$. From Fig. 6, it is possible to observe that PID, DFNN and DNN_0 cannot completely compensate for the disturbance. At the same time, DNN can learn the new conditions in the environment and obtain good performances.

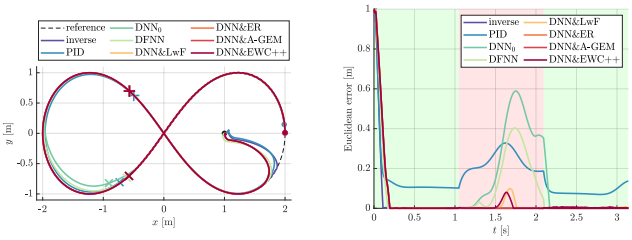
4) Dynamics with Noisy Measurements: In this scenario, the controllers are tested on a unicycle whose localisation accuracy is corrupted by a zero-mean white noise with a standard deviation of 0.1m for position and 0.1rad for heading. Fig. 7 shows that the PID controller is sensitive to noise due to the numerical time derivatives. At the same time, DNN_0 is quite tolerant to noisy inputs, and the DFNN-based controller shows good filtering capabilities thanks to its Gaussian fuzzification membership functions.



(a) Trajectory tracking: the coloured points indicate the final position reached by each controller.

(b) Tracking error.

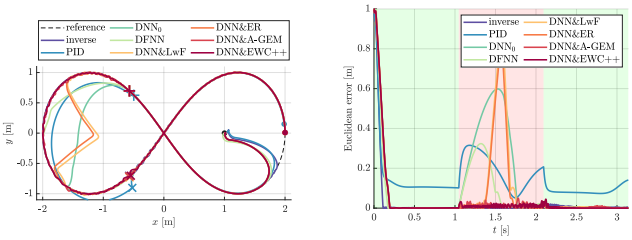
Fig. 4. Performances on the nominal unicycle.



(a) Trajectory tracking: the instance when uncertainty occurs is represented by $+$, while the instance when uncertainty vanishes is represented by $\times$.

(b) Tracking error: the green-shaded areas represent the period without uncertainty, while the red-shaded area represents the period with uncertainty.

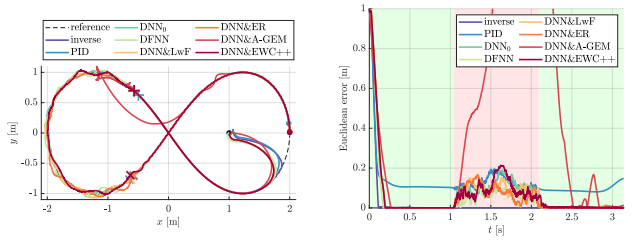
Fig. 5. Performances on the unicycle with internal uncertainties.



(a) Trajectory tracking: the instance when disturbance starts is represented by $+$, while the instance when disturbance ceases is represented by $\times$.

(b) Tracking error: the green-shaded areas represent the period without disturbance, while the red-shaded area is the period where disturbance is active.

Fig. 6. Performances on the unicycle with external disturbance.



(a) Trajectory tracking: the instance when the noise starts is represented by $+$, while the instance when noise is cancelled is represented by $\times$.

(b) Tracking error: the green-shaded areas represent the period without noise, while the red-shaded area is the period where noise is active.

Fig. 7. Performances on the unicycle with noisy measurements.

5) Discussion: In Table I, five tested controllers are compared under four different conditions in terms of the 2D tracking mean absolute error (MAE):

$$\text{MAE} = \frac{1}{n}\sum_{k=1}^{n} \|\mathbf{p}^*[k] - \mathbf{p}[k]\|_2, \tag{20}$$

where n is the number of sampled positions, $\mathbf{p}^*[k] = \begin{bmatrix} x^*[k] & y^*[k] \end{bmatrix}^T$ and $\mathbf{p}[k] = \begin{bmatrix} x[k] & y[k] \end{bmatrix}^T$ are the desired and actual positions at discrete time k, respectively. Table I shows that the exact analytical inverse of the robot's dynamics can track the reference trajectory with a small error caused by the saturated control inputs and initial transition phase to reach the path. On the other hand, the DNN-based controller outperforms other tested controllers in cases of uncertainty and disturbance. In the case of noisy measurements, DNN tries to learn the noise which might cause unstable behaviour. This issue can be alleviated by processing input data by a filter, e.g. a low-pass filter. Still, due to its intrinsic gradient calculation, A-GEM is extremely sensitive to noisy samples, which makes this method unstable with higher noise levels. To compare the trajectory tracking performances at steady-state of eight tested controllers in four different scenarios, a boxplot is presented in Fig. 8. To prove that DNN with CL remembers previously seen conditions, we repeat the same scenarios for the second time with the models saved after the first iteration. The performances of DNN enhanced with CL methods are provided in Table II. It was observed that when DNN faces the conditions previously seen, CL leads to a faster convergence time.

B. Dynamical Simulation

Research and development in aerial robotics have been growing unprecedentedly in recent years. Unmanned aerial vehicles (UAVs) define the family of the most agile and widely-used mobile robots. The position of UAV is $\mathbf{p} = \begin{bmatrix} x & y & z \end{bmatrix}^T$, its attitude is $\begin{bmatrix} \phi & \theta & \psi \end{bmatrix}^T$, its linear velocity is $\begin{bmatrix} v_x & v_y & v_z \end{bmatrix}^T$, its angular rates are $\begin{bmatrix} \omega_\phi & \omega_\theta & \omega_\psi \end{bmatrix}^T$. The UAV is controlled by adjusting the overall thrust T and the moments τ_ϕ, τ_θ and τ_ψ. The dynamical model of UAV is derived in [30], and it can be reworked in form of (1), where $\mathbf{x} = \begin{bmatrix} x & y & z & \phi & \theta & \psi & v_x & v_y & v_z & \omega_\phi & \omega_\theta & \omega_\psi \end{bmatrix}^T$, $\mathbf{u} = \begin{bmatrix} T & \tau_\phi & \tau_\theta & \tau_\psi \end{bmatrix}^T$, $\mathbf{y} = \begin{bmatrix} x & y & z & \psi \end{bmatrix}$, $f(\mathbf{x})$ and $g(\mathbf{x})$ are defined in [31]. If the low-level attitude controller is included in the UAV's dynamics, as in [32], then the virtual control inputs are $\mathbf{u} = \begin{bmatrix} v_z & \phi & \theta & \omega_\psi \end{bmatrix}^T$.

Remark 5: Only some basic properties of UAV's model are known, such as the number of control inputs $N_I = 4$ and observed outputs $N_O = 4$, and the relative degree $\mathbf{r} = [1, 2]$.

For testing the developed controllers, they were implemented in the robot operating system (ROS) and tested in the Gazebo simulation environment using a developed middleware package. The desired position of the drone $\mathbf{p}^*[k] = \begin{bmatrix} x^*[k] & y^*[k] & z^*[k] \end{bmatrix}^T$ is a time-based closed-loop 3-dimensional ∞-shaped trajectory:

$$\begin{cases} x^*[k] &= 40\frac{\cos(\frac{2}{5}k)}{3-\cos(\frac{4}{5}k)} \\ y^*[k] &= -20\frac{\sin(\frac{4}{5}k)}{3-\cos(\frac{4}{5}k)} \\ z^*[k] &= -20\frac{\sin(\frac{4}{5}k)}{3-\cos(\frac{4}{5}k)} + 10. \end{cases} \tag{21}$$

This trajectory contains most manoeuvres during a real flight mission, such as ascending and descending actions, straight segments and curved sections with left-hand and right-hand turns. From (21), it can be calculated that the desired linear velocity is variable between 8.5m/s and 14.1m/s.

To collect samples for the offline pre-training, the drone is controlled by a well-tuned hierarchical PID controller. One million instances $\{x[k], y[k], z[k], \psi[k], v_z[k], \phi[k], \theta[k], \omega_\psi[k]\}$

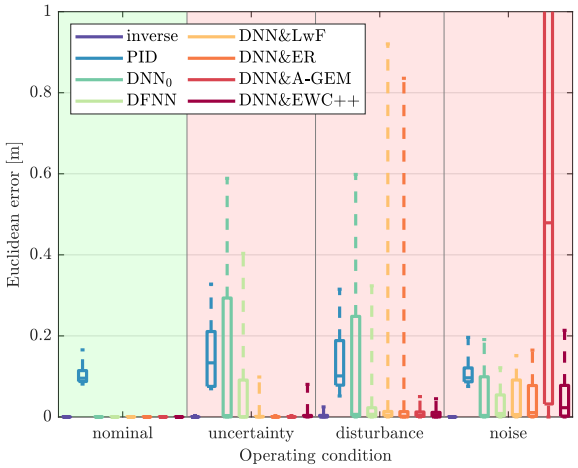


Fig. 8. Tracking performance of eight tested controllers in four different scenarios.

TABLE I
COMPARISON OF DIFFERENT CONTROLLERS FOR TRAJECTORY TRACKING OF UNICYCLE UNDER DIFFERENT OPERATION CONDITIONS IN TERMS OF POSITION MEAN ABSOLUTE ERROR (MAE) [m].

Controller	Nominal dynamics	Dynamics with uncertainty	Dynamics with disturbance	Dynamics with noise (average)
Inverse	0.020	0.020	0.022	0.020
PID	0.123	0.159	0.147	0.126
DNN$_0$ [27]	0.032	0.123	0.122	0.067
DFNN [29]	0.028	0.078	0.063	0.048
DNN&LwF [14]	0.031	0.036	0.086	0.063
DNN&ER [18]	0.031	0.032	0.077	0.054
DNN&A-GEM [20]	0.032	0.033	0.043	0.543
DNN&EWC++ [21]	0.031	0.036	0.036	0.065

TABLE II
COMPARISON OF DIFFERENT CL TECHNIQUES FOR THE SECOND ITERATION UNDER THE SAME CONDITIONS AS IN TABLE I IN TERMS OF POSITION MEAN ABSOLUTE ERROR (MAE) [m].

Controller	Nominal dynamics	Dynamics with uncertainty	Dynamics with disturbance	Dynamics with noise (average)
DNN&LwF [14]	0.031	0.032	0.045	0.063
DNN&ER [18]	0.031	0.032	0.077	0.054
DNN&A-GEM [20]	0.032	0.033	0.043	0.554
DNN&EWC++ [21]	0.031	0.032	0.036	0.065

are collected. To prepare the training samples, the linear velocities $v_x[k]$ and $v_y[k]$ are numerically derived from the positions. Besides, physics-informed transformations for position and attitude are performed similarly to the unicycle. Consequently, the input-output training pairs $\mathcal{D}_0[k] =< \{\cos(\psi[k]), \sin(\psi[k]), v_x[k], v_y[k], x[k+2] - x[k], y[k+2] - y[k], z[k+1] - z[k], \psi[k+1] - \psi[k]\}, \{v_z[k], \phi[k], \theta[k], \omega_\psi[k]\} >$ are built like in (9). The network consists of two fully connected hidden layers with 128 neurons in each layer besides the input and output layers. Then, DNN_0 is trained on $\mathcal{D}_0$ using the MSE loss function and Adam optimizer. After offline training, DNN through the feedforward input $\mathcal{I}[k] = \{\cos(\psi[k]), \sin(\psi[k]), v_x[k], v_y[k], x[k+2]^* - x[k], y^*[k+2] - y[k], z^*[k+1] - z[k], \psi^*[k+1] - \psi[k]\}$ generates the control signal $\mathcal{O}[k] = \{v_z[k], \phi[k], \theta[k], \omega_\psi[k]\}$ to reach the desired pose $\{x^*[k+2], y^*[k+2], z^*[k+1], \psi^*[k+1]\}$. At each iteration, the cache is updated with $\mathcal{D}[k] =< \{\cos(\psi[k-2]), \sin(\psi[k-2]), v_x[k-2], v_y[k-2], x[k] - x[k-2], y[k] - y[k-2], z[k] - z[k-1], \psi[k] - \psi[k-1]\}, \{v_z[k-1], \phi[k-2], \theta[k-2], \omega_\psi[k-1]\} >$.

To show the efficiency of the presented DNN-based controllers enhanced with CL techniques, their performances are compared with the performances of the baseline DNN_0 controller, the exact analytical inverse, a well-tuned PID controller (used for collecting training samples) and deep fuzzy neural network (DFNN)-based controller from [29]. These controllers are compared in a combined scenario where several variations to the nominal dynamics occur. First, white noise with a standard deviation of 0.1m is injected into the localisation system of the drone. Then, uncertainty occurs: the mass and, consequently, all moments of inertia are doubled. Finally, the disturbance is introduced: a wind gust starts blowing from north-west-up at $4.2\text{m}\,\text{s}^{-1}$. Subsequently, all modifications to the dynamics are removed in the inverse order: first, disturbance; then, uncertainty; and finally, noise.

Fig. 9 shows the 3D tracking performances of the controllers under examination. The trajectory tracking is projected on x-, y- and z-axes in Fig. 10a. From Fig. 10b, it is possible to observe the evolution of the Euclidean error between desired and actual positions. The PID controller has acceptable performances thanks to the low-level UAV's controller, but it always lags behind the desired position as being an error-driven controller. At the same time, the DNN_0-based controller performs well on the nominal dynamics, while it suffers when the dynamics is altered. On the other hand, it is possible to observe the superiority of DNN-based controllers with learning capabilities under various operating conditions. Nevertheless, controllers with CL have some oscillatory behaviour under noisy conditions since training is sensitive to the samples with noise. This issue can be alleviated by processing input data by a filter, e.g. a low-pass filter. When noise vanishes, DNN learns the nominal model again and stabilises the robot. Interestingly, when the mass of the drone increases, the oscillatory behaviour disappears due to the increased moments of inertia. Table III compares the tested controllers with respect to the most critical metrics for autonomous and mobile robots: average computation time, required memory space and trajectory tracking accuracy,

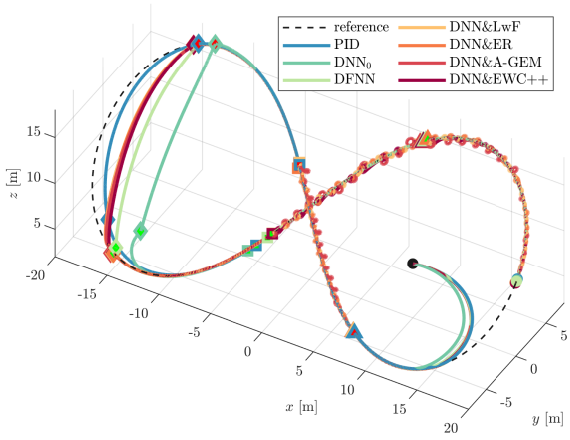
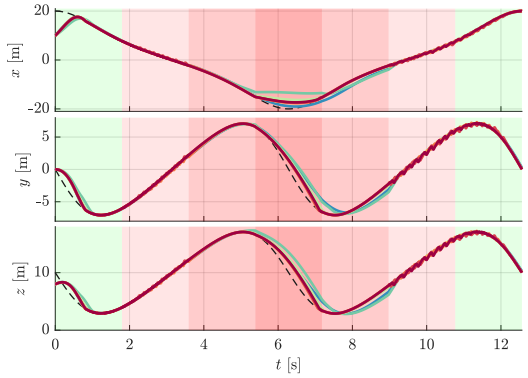
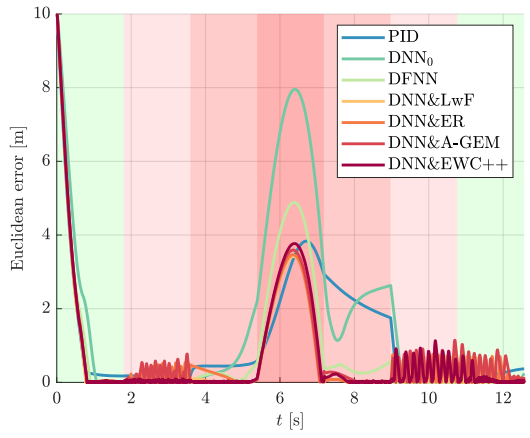


Fig. 9. Performances of various controllers on the drone affected by the following conditions: noisy measurements (start ▲ and end ▲ locations), internal uncertainties (start ■ and end ■ locations), and external disturbance (start ♦ and end ♦ locations).



(a) Trajectory tracking projection on x-, y- and z-axes.



(b) Mean Euclidean error between desired and actual positions.

Fig. 10. Performances of different controllers on the drone under the following conditions: the green-shaded areas represent nominal conditions, while the red-shaded areas represent conditions with various levels of modifications to the drone's dynamics (lighter-red areas – noise, medium-red areas – noise & uncertainty, darker-red area – noise & uncertainty & disturbance).

TABLE III
COMPARISON OF DIFFERENT CONTROLLERS FOR UAV TRAJECTORY TRACKING IN TERMS OF POSITION TRACKING MAE, COMPUTATIONAL TIME AND REQUIRED MEMORY SPACE.

Controller	Tracking Error [m]		Computation Time [ms]	Memory Space [kB]
	First run	Second run		
PID	1.137		0.073	~ 0
DNN$_0$ [27]	1.632		0.284	≈ 71
DFNN [29]	0.846	0.803	0.878	≈ 212
DNN&LwF [14]	0.659	0.475	3.756	≈ 308
DNN&ER [18]	0.712	0.506	4.520	≈ 309
DNN&A-GEM [20]	0.752	0.663	5.112	≈ 310
DNN&EWC++ [21]	0.693	0.461	3.824	≈ 215

defined as a 3D mean absolute error:

$$\text{MAE} = \frac{1}{n}\sum_{k=1}^{n} \|\mathbf{p}^*[k] - \mathbf{p}[k]\|_2, \tag{22}$$

where n is the number of sampled positions. The workstation on which the algorithms were executed has 14 cores CPU, 32GB RAM and 16GB VRAM. The proposed DNN controllers with CL capabilities have the best tracking accuracy, especially during the second iteration over the trajectory, but at the cost of increased computation time and required memory space. Generally, it is possible to observe the trend that more advanced controllers require more processing power and storage space but have improved performances.

VI. CONCLUSIONS AND FUTURE WORK

In this work, we validated online CL for the control of robotic systems which improves robot performance in real-time. The proposed approach fuses deep learning, online learning and CL by adopting the advantages of each. The learning is divided into two stages: offline and online training. In the offline phase, a conventional PID controller performs a set of random manoeuvres to collect a batch of input-output training samples from the robot. Then, a DNN-based controller, DNN$_0$, is pre-trained offline on the collected dataset to approximate the inverse dynamics of the robot. Nevertheless, DNN$_0$ is a static controller which cannot adapt to new operating conditions. In the online phase, the online learning DNN-based controller, DNN, takes over the control of the robot, adapts in real-time to the current conditions thanks to online CL and improves the tracking performance. The simulation results demonstrate that the proposed DNN methods improve tracking accuracy. We believe that the outcomes of this study will open doors to the broader use of DNN-based controllers with online CL for a variety of applications in autonomous and mobile robotics.

In the future, we will validate the proposed online continual DNN-based control approach on real-world robots which are affected by numerous second-order effects difficult to model. Moreover, we will investigate the influence of CL on control stability during learning. In addition, we will study the effects of the parameters in CL methods on online learning efficiency.

REFERENCES

[1] L. Tai and M. Liu, "Deep-learning in Mobile Robotics - from Perception to Control Systems: A Survey on Why and Why not," *CoRR*, vol. abs/1612.07139, 2016.

[2] H. Shakhatreh, A. H. Sawalmeh, A. Al-Fuqaha, Z. Dou, E. Almaita, I. Khalil, N. S. Othman, A. Khreishah, and M. Guizani, "Unmanned Aerial Vehicles (UAVs): A Survey on Civil Applications and Key Research Challenges," *IEEE Access*, vol. 7, pp. 48 572–48 634, 2019.

[3] M. V. Butz, D. Bilkey, D. Humaidan, A. Knott, and S. Otte, "Learning, planning, and control in a monolithic neural event inference architecture," *Neural Networks*, vol. 117, pp. 135–144, 2019.

[4] R. K. Srivastava, K. Greff, and J. Schmidhuber, "Training Very Deep Networks," in *Proceedings of the 28th International Conference on Neural Information Processing Systems - Volume 2*, ser. NIPS'15. Cambridge, MA, USA: MIT Press, 2015, p. 2377–2385.

[5] S. C. Hoi, D. Sahoo, J. Lu, and P. Zhao, "Online learning: A comprehensive survey," *Neurocomputing*, vol. 459, pp. 249–289, 2021.

[6] S. Patel, A. Sarabakha, D. Kircali, and E. Kayacan, "An Intelligent Hybrid Artificial Neural Network-Based Approach for Control of Aerial Robots," *Journal of Intelligent & Robotic Systems*, vol. 97, no. 2, pp. 387–398, Feb 2020.

[7] S. Zhou, A. Sarabakha, E. Kayacan, M. K. Helwa, and A. P. Schoellig, "Knowledge Transfer Between Robots with Similar Dynamics for High-Accuracy Impromptu Trajectory Tracking," in *2019 European Control Conference (ECC)*, June 2019, pp. 1–8.

[8] A. Sarabakha and E. Kayacan, "Online Deep Learning for Improved Trajectory Tracking of Unmanned Aerial Vehicles Using Expert Knowledge," in *2019 International Conference on Robotics and Automation (ICRA)*, 2019, pp. 7727–7733.

[9] Z. Li and D. Hoiem, "Learning without Forgetting," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 12, pp. 2935–2947, 2018.

[10] E. Belouadah, A. Popescu, and I. Kanellos, "A comprehensive study of class incremental learning algorithms for visual tasks," *Neural Networks*, vol. 135, pp. 38–54, 2021.

[11] G. I. Parisi, R. Kemker, J. L. Part, C. Kanan, and S. Wermter, "Continual lifelong learning with neural networks: A review," *Neural Networks*, vol. 113, pp. 54–71, 2019.

[12] T. Lesort, V. Lomonaco, A. Stoian, D. Maltoni, D. Filliat, and N. Díaz-Rodríguez, "Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges," *Information Fusion*, vol. 58, pp. 52–68, 2020.

[13] A. Cossu, A. Carta, V. Lomonaco, and D. Bacciu, "Continual learning for recurrent neural networks: An empirical evaluation," *Neural Networks*, vol. 143, pp. 607–627, 2021.

[14] Z. Li and D. Hoiem, "Learning Without Forgetting," in *Computer Vision – ECCV 2016*, B. Leibe, J. Matas, N. Sebe, and M. Welling, Eds. Cham: Springer International Publishing, 2016, pp. 614–629.

[15] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming catastrophic forgetting in neural networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[16] V. Lomonaco, L. Pellegrini, A. Cossu, A. Carta, G. Graffieti, T. L. Hayes, M. De Lange, M. Masana, J. Pomponi, G. M. van de Ven, M. Mundt, Q. She, K. Cooper, J. Forest, E. Belouadah, S. Calderara, G. I. Parisi, F. Cuzzolin, A. S. Tolias, S. Scardapane, L. Antiga, S. Ahmad, A. Popescu, C. Kanan, J. van de Weijer, T. Tuytelaars, D. Bacciu, and D. Maltoni, "Avalanche: An End-to-End Library for Continual Learning," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, June 2021, pp. 3600–3610.

[17] Z. Mai, R. Li, J. Jeong, D. Quispe, H. Kim, and S. Sanner, "Online continual learning in image classification: An empirical survey," *Neurocomputing*, vol. 469, pp. 28–51, 2022.

[18] A. Chaudhry, M. Rohrbach, M. Elhoseiny, T. Ajanthan, P. K. Dokania, P. H. S. Torr, and M. Ranzato, "Continual Learning with Tiny Episodic Memories," *CoRR*, vol. abs/1902.10486, 2019.

[19] Z. Mai, R. Li, J. Jeong, D. Quispe, H. Kim, and S. Sanner, "Online continual learning in image classification: An empirical survey," *Neurocomputing*, vol. 469, pp. 28–51, 2022.

[20] A. Chaudhry, M. Ranzato, M. Rohrbach, and M. Elhoseiny, "Efficient Lifelong Learning with A-GEM," in *International Conference on Learning Representations*, 2019.

[21] A. Chaudhry, P. K. Dokania, T. Ajanthan, and P. H. S. Torr, "Riemannian Walk for Incremental Learning: Understanding Forgetting and Intransigence," in *Computer Vision – ECCV 2018*, V. Ferrari, M. Hebert, C. Sminchisescu, and Y. Weiss, Eds. Cham: Springer International Publishing, 2018, pp. 556–572.

[22] C. S. Yin Wong, G. Yang, A. Ambikapathi, and R. Savitha, "Online Continual Learning Using Enhanced Random Vector Functional Link Networks," in *ICASSP 2022 - 2022 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2022, pp. 1905–1909.

[23] M. De Lange, R. Aljundi, M. Masana, S. Parisot, X. Jia, A. Leonardis, G. Slabaugh, and T. Tuytelaars, "A Continual Learning Survey: Defying Forgetting in Classification Tasks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3366–3385, 2022.

[24] Y. He and B. Sick, "CLeaR: An adaptive continual learning framework for regression tasks," *AI Perspectives*, vol. 3, no. 1, p. 2, Jul 2021.

[25] J. Fernández de Cañete, C. Galindo, and I. G. Moral, *System Description*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 43–83.

[26] A. Isidori, *Elementary Theory of Nonlinear Feedback for Multi-Input Multi-Output Systems*. London: Springer London, 1995, pp. 219–291.

[27] S. Zhou, M. K. Helwa, and A. P. Schoellig, "Design of deep neural networks as add-on blocks for improving impromptu trajectory tracking," in *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*, Dec 2017, pp. 5201–5207.

[28] S. Mohan, J. Nandagopal, and S. Amritha, "Decoupled Dynamic Control of Unicycle Robot Using Integral Linear Quadratic Regulator and Sliding Mode Controller," *Procedia Technology*, vol. 25, pp. 84–91, 2016.

[29] A. Sarabakha and E. Kayacan, "Online Deep Fuzzy Learning for Control of Nonlinear Systems Using Expert Knowledge," *IEEE Transactions on Fuzzy Systems*, vol. 28, no. 7, pp. 1492–1503, 2020.

[30] R. Mahony, V. Kumar, and P. Corke, "Multirotor Aerial Vehicles: Modeling, Estimation, and Control of Quadrotor," *Robotics Automation Magazine, IEEE*, vol. 19, no. 3, pp. 20–32, 2012.

[31] A. Sarabakha and E. Kayacan, "Y6 Tricopter Autonomous Evacuation in an Indoor Environment Using Q-Learning Algorithm," in *2016 IEEE 55th Conference on Decision and Control (CDC)*, Dec 2016, pp. 5992–5997.

[32] T. Lee, M. Leok, and N. H. McClamroch, "Nonlinear Robust Tracking Control of a Quadrotor UAV on SE(3)," *Asian Journal of Control*, vol. 15, no. 2, pp. 391–408, 2013.